Security Audit Report

Neutron Auction and Undelegations Contract, Chain Upgrade

Phase 1

v1.0

April 9, 2025

Table of Contents

Table of Contents 2
License 3
Disclaimer 4
Introduction 5
Purpose of This Report 5
Codebase Submitted for the Audit 5
Methodology 6
Functionality Overview 6
How to Read This Report 7
Code Quality Criteria 8
Summary of Findings 9
Detailed Findings 10
1. USDC budget is declared but never verified to be held by the contract 10
2. Unvalidated auction_end_time allows silently closing or reopening deposits 11
3. Owner can reduce total_usdc_budget to zero after users have committed funds they cannot withdraw 11
4. Burning an IBC voucher does not burn the underlying tokens in escrow on the source chain 12
5. Changing claim_token_denom after deposits corrupts accounting and permanently bricks settlement 12
6. Leftover USDC due to rounding will be irrecoverable 13
7. README instantiation example contains incorrect and missing fields 14
8. README misleads users about when claiming becomes available 14
9. Direct ownership transfer allows accidental loss of contract control 15
10. instantiate and update_config emit insufficient event attributes 15
11. Non-claim_token_denom funds attached to Deposit are silently locked forever 16
12. Deposit accumulation uses unchecked addition 16
13. InsufficientFunds error variant is defined but never used 17
14. Contract name in cw2 metadata does not match the crate name 17
15. Zero-share depositors have their tokens burned with no USDC received and no notification 18

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Hadron Labs to perform a security audit of Neutron Auction and Undelegations Contract, Chain Upgrade.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/neutron-org/auction
Commit	`5b463b1a5a2f86ef739d3b6a06e8906649b40fe4`
Scope	The scope is restricted to the auction contract.
Fixes verified at commit	`9cbad3c4522b4e063ac0aa0918bc3c9b34f0144a` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

The auction contract implements a time-bounded deposit auction in which users deposit a specified coin, tracking per-address deposits and the total amount deposited.

After the auction ends, the contract owner can run settlements in batches, proportionally allocating a fixed USDC budget across all participants based on their share of total deposits, recording each user's USDC claim, and burning all deposited claim tokens once settlement is complete.

Once settlement is marked finished, users can claim their allocated USDC, which is sent directly from the contract to their address, and their claim record is cleared.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**Low-Medium**	The codebase adheres to established language and framework best practices, and the business logic is clearly defined and appropriately scoped.
Code readability and clarity	**Medium-High**	-
Level of documentation	**Medium**	-
Test coverage	**Medium-High**	Test coverage reported by `cargo tarpaulin` is `92.44%`

Summary of Findings

No	Description	Severity	Status
1	USDC budget is declared but never verified to be held by the contract	Minor	Acknowledged
2	Unvalidated `auction_end_time` allows silently closing or reopening deposits	Minor	Resolved
3	Owner can reduce `total_usdc_budget` to zero after users have committed funds they cannot withdraw	Minor	Resolved
4	Burning an IBC voucher does not burn the underlying tokens in escrow on the source chain	Minor	Acknowledged
5	Changing `claim_token_denom` after deposits corrupts accounting and permanently bricks settlement	Minor	Resolved
6	Leftover USDC due to rounding will be irrecoverable	Minor	Resolved
7	README instantiation example contains incorrect and missing fields	Informational	Resolved
8	README misleads users about when claiming becomes available	Informational	Resolved
9	Direct ownership transfer allows accidental loss of contract control	Informational	Acknowledged
10	`instantiate` and `update_config` emit insufficient event attributes	Informational	Resolved
11	Non-`claim_token_denom` funds attached to `Deposit` are silently locked forever	Informational	Resolved
12	Deposit accumulation uses unchecked addition	Informational	Resolved
13	InsufficientFunds error variant is defined but never used	Informational	Resolved
14	Contract name in cw2 metadata does not match the crate name	Informational	Resolved
15	Zero-share depositors have their tokens burned with no USDC received and no notification	Informational	Resolved

Detailed Findings

1. USDC budget is declared but never verified to be held by the contract

Severity: Minor

In `contracts/auction/src/contract.rs:21-52`, `instantiate` accepts `total_usdc_budget` as a plain configuration value but does not require the instantiator to attach the corresponding `usdc_denom` funds. The contract has no dedicated mechanism for the owner to deposit USDC: there is no `DepositUsdcFunds` message or equivalent. The only path by which USDC can enter the contract is via a direct bank transfer to the contract address, which is entirely outside the contract's control and tracking.

As a result, the contract makes no guarantee at any point (not at instantiation, not at the start of settlement, and not at claim time) that it actually holds the `total_usdc_budget` it promises to distribute. If the operator fails to send USDC to the contract address, or sends an insufficient amount, the `execute_claim` call may fail when `BankMsg::Send` is executed in `contract.rs:261-267` if there is not enough funds to send.

This also means `total_usdc_budget` in config and the actual USDC balance of the contract are two independent values that can silently diverge. The declared budget can be reduced via `UpdateConfig` without any effect on the real balance, and the real balance can be increased or decreased by direct transfers without updating the config.

Recommendation

We recommend requiring the owner to attach `total_usdc_budget` of `usdc_denom` funds when calling `instantiate`, and verifying that `info.funds` contains exactly that amount.

Alternatively, we recommend adding a dedicated `DepositUSDCBudget` execute message that accepts `usdc_denom` funds and increases `total_usdc_budget` by the attached amount, making the contract the authoritative source of truth for the distributable budget.

Status: Acknowledged

2. Unvalidated `auction_end_time` allows silently closing or reopening deposits

Severity: Minor

In `contracts/auction/src/contract.rs:226-228` and in instantiation in line `29`, `auction_end_time` is accepted without any validation. Setting it to a past timestamp immediately closes the deposit window.

More notably, because `UpdateConfig` is only gated on `last_address.is_none() && !is_finished`, the owner can reset a past end time to a future value in the window between the deposit period ending and the first settlement batch running, effectively reopening deposits. Original depositors would have their shares silently diluted by the new deposits.

Recommendation

We recommend validating that `auction_end_time` is strictly in the future at both instantiation and in `UpdateConfig`, and preventing it from being changed once the original end time has elapsed.

Status: Resolved

3. Owner can reduce `total_usdc_budget` to zero after users have committed funds they cannot withdraw

Severity: Minor

In `contracts/auction/src/contract.rs:235-237`, the `UpdateConfig` function allows the owner to set `total_usdc_budget` to any value, including zero, as long as no settlement batch has started. Since there is no withdrawal mechanism in the contract, users who have already deposited `claim_token_denom` cannot retrieve their tokens.

If the owner reduces `total_usdc_budget` to zero before settlement, the pro-rata share formula yields zero for every depositor, no `CLAIMS` entries are written, and all deposited tokens are burned at the end of settlement. Users permanently lose their deposited tokens and receive nothing in return.

We classify this issue as minor since the `UpdateConfig` function can only be called by the owner.

Recommendation

We recommend only allowing `total_usdc_budget` to be increased, never decreased, at least once the first deposit has been made, given that users are committing funds with an expectation of the announced budget.

Status: Resolved

4. Burning an IBC voucher does not burn the underlying tokens in escrow on the source chain

Severity: Minor

In `contracts/auction/src/contract.rs:185-190`, the final settlement batch emits a `BankMsg::Burn` for the full `total_deposited` amount of `claim_token_denom`.

If `claim_token_denom` is an IBC-bridged token, burning the voucher does not trigger a burn of the original tokens on the source chain: it only destroys the local representation. The tokens that were escrowed on the source chain when the IBC transfer was originally made remain permanently locked there.

We classify this issue as minor since the `claim_token_denom` value can only be set by the owner at instantiation or in the `UpdateConfig` function.

Recommendation

We recommend restricting `claim_token_denom` to tokenfactory or native denoms only, rejecting IBC denoms at instantiation and in `UpdateConfig`.

Status: Acknowledged

5. Changing `claim_token_denom` after deposits corrupts accounting and permanently bricks settlement

Severity: Minor

In `contracts/auction/src/contract.rs:229-231`, the `UpdateConfig` function allows the owner to change `claim_token_denom` at any point before the first settlement batch, even after users have already deposited under the original denom. This creates several compounding issues.

First, the `TOTAL_DEPOSITED` counter and all `DEPOSITS` map entries remain unchanged after the denom change. They reflect amounts of the original token. Users can deposit the new token and their amounts are added to `TOTAL_DEPOSITED` as if they are the same unit

as the original token. The share computation during settlement then mixes amounts of two different tokens in the pro-rata formula, producing economically meaningless results.

Second, and more critically, the final settlement batch emits `BankMsg::Burn { denom: claim_token_denom, amount: total_deposited }` where `total_deposited` includes the sum of both old and new token amounts, but the contract only holds the new token in the amount deposited under the new denom. The burn attempts to burn more of the new token than the contract holds, which would fail. Settlement can never complete, `is_finished` is never set to `true`, and all `CLAIMS` entries written in earlier batches are permanently unclaimable. Original depositors also cannot retrieve their tokens since there is no withdrawal path.

We classify this issue as minor since the `UpdateConfig` function can only be called by the owner.

Recommendation

We recommend preventing `claim_token_denom` from being changed once the first deposit has been made. This can be enforced by checking that `TOTAL_DEPOSITED` is zero before allowing the denom field to be updated in `UpdateConfig`.

Status: Resolved

6. Leftover USDC due to rounding will be irrecoverable

Severity: Minor

In `contracts/auction/src/contract.rs:159-168`, the `execute_settle` function computes the user's share portion from the USDC budget based on their deposited amount. The formula used is `deposit * total_usdc_budget / total_deposited`.

Over time, rounding issues may leave some USDC in the contract, which will not be distributed to users.

Recommendation

We recommend implementing a `sweep` function that the owner can call after all claims are processed, and sending the remaining USDC to `dao_treasury`.

The `dao_treasury` is mentioned in `README.md:55`, but it was never implemented in the contract, as noted in this finding.

Status: Resolved

7. README instantiation example contains incorrect and missing fields

Severity: Informational

In `README.md:51-57`, the example JSON for instantiation shows fields that do not correspond to the actual `InstantiateMsg` struct. The example includes `auction_duration_seconds` and `dao_treasury`, which do not exist in the struct, and omits the required `owner` and `auction_end_time` fields. The actual struct is defined in `contracts/auction/src/msg.rs:5-11`.

This inconsistency may confuse integrators attempting to instantiate the contract from the documentation.

Recommendation

We recommend updating the README example to reflect the actual `InstantiateMsg` fields.

Status: Resolved

8. README misleads users about when claiming becomes available

Severity: Informational

In `README.md:93`, the claim section states that users can claim their USDC share *"after their deposit has been processed in settlement."*

This implies that a user may claim as soon as their individual deposit is settled in a batch. In reality, `execute_claim` requires `progress.is_finished == true`, meaning the entire settlement must complete across all batches before any user can claim.

A user whose deposit was processed in the first batch still cannot claim until the final batch finishes.

Recommendation:

We recommend updating the README to clarify that claiming is only possible after the full settlement process is complete, not after an individual batch processes a given depositor.

Status: Resolved

9. Direct ownership transfer allows accidental loss of contract control

Severity: Informational

In `contracts/auction/src/contract.rs:223-225`, the `UpdateConfig` function allows the owner to transfer ownership to a new address in a single step by passing a new value in the `owner` field. While this is common practice, it presents a risk for the ownership of the contract to become lost if the owner transfers ownership to an incorrect address.

A two-step ownership transfer will allow the current owner to propose a new owner, and then the account that is proposed as the new owner may call a function that will allow them to claim ownership and actually execute the config update.

Recommendation

We recommend implementing a two-step ownership transfer. The flow can be as follows:

1. The current owner proposes a new owner address that is validated and lowercased.
2. The new owner account claims ownership, which applies the configuration changes.

Status: Acknowledged

10. `instantiate` and `update_config` emit insufficient event attributes

Severity: Informational

In `contracts/auction/src/contract.rs:49-51`, the `instantiate` entry point only emits `action` and `auction_end_time` as response attributes. Important configuration parameters such as `owner`, `claim_token_denom`, `usdc_denom`, and `total_usdc_budget` are not emitted, making it harder to verify the initial contract configuration from transaction history alone.

Furthermore, in `contracts/auction/src/contract.rs:241`, the `UpdateConfig` handler returns a response with only the `action=update_config` attribute. None of the updated field values are emitted. This makes it impossible to determine what changed in a given configuration update from transaction history.

Recommendation

We recommend emitting all relevant configuration fields as attributes in the `instantiate` and `update_config` responses.

Status: Resolved

11. Non-`claim_token_denom` funds attached to `Deposit` are silently locked forever

Severity: Informational

In `contracts/auction/src/contract.rs:90-95`, `execute_deposit` iterates `info.funds` to find the coin matching `claim_token_denom` and silently ignores all other coins. Any other tokens attached to the same transaction are not returned to the user, so they will be stuck in the contract forever.

While blockchains generally do not protect users from sending funds to wrong accounts, reverting extra funds increases the user experience.

Recommendation

We recommend checking that the transaction contains only the expected `Coin` and no additional native tokens using https://docs.rs/cw-utils/latest/cw_utils/fn.must_pay.html.

Status: Resolved

12. Deposit accumulation uses unchecked addition

Severity: Informational

In `contracts/auction/src/contract.rs:102`, the global deposit counter is incremented with the += operator: `total_deposited += deposit_amount`.

Using `checked_add` makes the potential overflow behavior explicit and returns a typed error that can be handled gracefully.

Recommendation

We recommend replacing `total_deposited += deposit_amount` with `total_deposited = total_deposited.checked_add(deposit_amount)?` to make overflow handling explicit and consistent with the rest of the codebase.

Status: Resolved

13. `InsufficientFunds` error variant is defined but never used

Severity: Informational

In `contracts/auction/src/error.rs:30-31`, the `InsufficientFunds` variant is declared in the `ContractError` enum but is never constructed or returned anywhere in the contract.

Dead error variants add noise to the error type and may mislead integrators into expecting conditions that are never enforced.

Recommendation

We recommend removing the `InsufficientFunds` variant, or implementing the corresponding balance check.

Status: Resolved

14. Contract name in `cw2` metadata does not match the crate name

Severity: Informational

In `contracts/auction/src/contract.rs:17`, `CONTRACT_NAME` is hardcoded as `"crates.io:claimer"`, while the actual crate is named `"auction"`.

The `cw2` standard stores this string on-chain via `set_contract_version` and is used by migration tooling to verify that a migration target is the correct contract. If a `migrate` entry point is added in the future and a migration guard checks the stored contract name, it will fail unless the mismatch is accounted for.

Recommendation

We recommend updating `CONTRACT_NAME` to `"crates.io:auction"` to match the actual crate name.

Status: Resolved

15. Zero-share depositors have their tokens burned with no USDC received and no notification

Severity: Informational

In `contracts/auction/src/contract.rs:166-168`, during settlement, a `CLAIMS` entry is only written when `user_share > 0`.

Users whose deposit is so small that `floor(deposit * total_usdc_budget / total_deposited) == 0` receive no entry and no event is emitted, yet their `claim_token_denom` tokens are included in the `BankMsg::Burn` at the end of settlement.

Recommendation

We recommend emitting an event for zero-share depositors during settlement to maintain an on-chain record that their deposit was processed.

Status: Resolved